

Project Title: Vision-Driven Cyber-Physical System for Automated Skin Disease Identification

1. Abstract

This project presents a novel, multi-modal Deep Learning system designed to automate the diagnosis of skin diseases. Unlike traditional Computer Vision models that rely solely on image data, this framework integrates **visual analysis** (dermoscopic images) with **clinical metadata** (patient symptoms) to simulate a dermatologist's diagnostic process. Utilizing a fused architecture comprising **EfficientNetB4** and a custom Dense Neural Network, the model achieves an overall accuracy of **96.08%** across 27 distinct disease classes. The system is deployed via a Flask web interface, allowing for real-time inference using a user-friendly questionnaire and image upload system.

2. Introduction

Skin diseases are among the most common human health afflictions, affecting millions globally. Accurate diagnosis requires significant expertise, often unavailable in remote or resource-constrained regions. While Computer Vision has made strides in detecting melanoma and other conditions, reliance on visual data alone often leads to misdiagnosis in cases where visual symptoms overlap (e.g., Eczema vs. Atopic Dermatitis).

In clinical practice, a diagnosis is rarely made by looking at a lesion in isolation. Doctors consider symptoms such as itching, bleeding, pain, and the lesion's evolution (chronic vs. new). This project aims to bridge the gap between AI and clinical reality by engineering a model that "sees" the lesion and "listens" to the patient's symptoms simultaneously.

3. Problem Statement

- **Visual Ambiguity:** Many skin conditions (e.g., fungal infections vs. psoriasis) manifest with visually similar rashes, making purely image-based classification prone to error.
- **Data Silos:** Most existing datasets and models ignore the rich metadata (symptoms) associated with skin lesions.
- **Accessibility:** High-performing diagnostic tools are often computationally expensive or inaccessible to non-experts.

4. Objectives

1. **Develop a Multi-Input Model:** Create a neural network architecture capable of processing image data and structured symptom vectors simultaneously.
2. **Maximize Classification Performance:** Achieve high precision and recall across a diverse set of 27 skin disease classes.
3. **Address Class Imbalance:** Implement strategies to handle disparities between common diseases (e.g., Melanocytic Nevi) and rare conditions.
4. **Deployment:** Create a lightweight, accessible web interface for end-users to interact with the model.

5. Architectural Overview

The core of the system is a **Multi-Input Model** built using TensorFlow/Keras. The architecture is

divided into two distinct branches that merge into a final classification head.

Branch A: The Visual Stream (Computer Vision)

- **Backbone: EfficientNetB4** (pre-trained on ImageNet). EfficientNet was selected for its superior balance between accuracy and computational efficiency compared to ResNet or VGG.
- **Input Shape:** (380, 380, 3). Images are resized to match the native resolution of EfficientNetB4.
- **Preprocessing:**
 - **Rescaling:** Pixel values are normalized to the [0, 1] range.
 - **Augmentation:** To prevent overfitting, the training pipeline includes random flips, rotations, zooms, and contrast adjustments.
- **Feature Extraction:** The top layers of EfficientNet are removed. A GlobalAveragePooling2D layer converts the feature map into a dense feature vector.

Branch B: The Symptom Stream (Clinical Data)

- **Input:** A 17-dimensional binary vector representing clinical symptoms (e.g., symptom_is_itchy, symptom_has_blisters, symptom_loc_face).
- **Structure:** A custom Feed-Forward Neural Network (Dense Network).
 - Layer 1: 128 Units (ReLU) + Dropout (0.2)
 - Layer 2: 256 Units (ReLU) + Dropout (0.2)
 - Layer 3: 512 Units (ReLU) + Dropout (0.2)
- **Purpose:** This branch learns the statistical correlation between specific symptoms and disease labels, acting as a "clinical verification" step.

Fusion and Output

- **Concatenation:** The output vectors from the Visual Branch (image embeddings) and the Symptom Branch (symptom embeddings) are concatenated into a single tensor.
- **Classification Head:**
 - Dense Layer (512 Units, ReLU) -> Dropout
 - Dense Layer (256 Units, ReLU) -> Dropout
 - Dense Layer (128 Units, ReLU) -> Dropout
 - **Output Layer:** A Dense layer with **27 units** and a **Softmax** activation function to output probability distributions for each class.

6. Tech Stack

- **Core Framework:** Python 3.12, TensorFlow 2.x, Keras 3.x.
- **Data Processing:** Pandas (metadata management), NumPy, OpenCV.
- **Deployment:** Flask (Web Framework), Hugging Face Hub (Model Hosting).
- **Version Control:** Git, Git LFS (for weight management).

7. The Dataset

The model was trained on the **Skin Disease Dataset** (sourced from Kaggle).

- **Visual Data:** Dermoscopic images organized into class-specific folders.
- **Metadata:** A CSV file mapping image IDs to symptom attributes.
- **Classes:** 27 distinct classes including Melanoma, Basal Cell Carcinoma, Eczema, and viral infections like Chickenpox and Monkeypox.

8. Training Configuration

- **Optimizer:** Adam Optimizer with a learning rate of 0.001.
- **Loss Function:** Categorical Crossentropy (suitable for multi-class classification).
- **Metrics:** Accuracy, Precision, Recall, AUC.
- **Callbacks:**
 - EarlyStopping: Monitors validation accuracy and halts training if no improvement is seen, preventing overfitting.
- **Environment:** Trained on NVIDIA GPUs (via Kaggle/Colab) to handle the computational load of EfficientNetB4.

9. Handling Model Persistence (The ".h5 vs .keras" Challenge)

During development, a version mismatch between Keras 2.x (Colab default) and Keras 3.x (Local Environment) caused serialization issues.

- **Issue:** The .keras format saved in the training environment created "ghost" input shapes (e.g., 381x381x1) when loaded locally.
- **Solution:** The model was explicitly reconstructed and saved using the legacy **.h5 (HDF5)** format. This ensured the architecture definitions (Input shape: 380x380x3) remained consistent across different TensorFlow versions.

10. Deployment Pipeline

The system is deployed using a modular Flask application:

1. **Frontend:** An HTML/Bootstrap interface allows users to upload an image and select symptoms via radio buttons.
2. **Backend (app.py):**
 - Dynamically downloads the model weights from **Hugging Face** (hf_hub_download) to bypass GitHub file size limits.
 - Constructs the model architecture in real-time.
 - Preprocesses inputs (resizes images, converts form data to binary vectors).
 - Returns the predicted class and confidence score.

11. Performance Metrics

The model was evaluated on a held-out test set comprising **12,791 images**.

- **Overall Accuracy: 96.08%**
- **Weighted Precision: 94.30%**
- **Weighted Recall: 96.08%**
- **Weighted F1-Score: 94.88%**

These metrics indicate a highly robust model capable of correct classification in the vast majority of cases.

12. Detailed Class Analysis

The classification report reveals performance disparities driven by **class imbalance**:

Top Performing Classes (Precision/Recall $\approx$ 1.00)

- **Basal Cell Carcinoma (Support: 2658):** The model performs flawlessly here. The high volume of training data and distinct visual features contribute to this success.
- **Melanocytic Nevi (Support: 6383):** Being the majority class, the model has learned to identify

these benign moles with near-perfect accuracy.

- **Melanoma (Support: 335):** Crucially, the model achieves 100% precision and recall on Melanoma, the most dangerous skin cancer. This is a vital success for a diagnostic tool.
- **Viral Infections (Chickenpox, Measles, Monkeypox):** These classes also show perfect scores, likely due to distinct symptom profiles (e.g., systemic spread) captured by the symptom branch.

Underperforming Classes (Precision < 0.60)

- **Atopic Dermatitis (Precision: 0.00, Support: 65):** The model failed to classify these cases correctly. This is likely due to the low sample size and visual similarity to Eczema.
- **Vascular Lesions (Precision: 0.00, Support: 26):** Extremely low support (only 26 test images) made it impossible for the model to generalize features for this class.
- **Benign Keratosis-like Lesions (Precision: 0.39):** While recall is high (1.00), precision is low, suggesting the model "over-predicts" this class (false positives), possibly confusing it with Seborrheic Keratoses.

13. Discussion on Imbalance

The dataset is heavily skewed. **Melanocytic Nevi** and **Basal Cell Carcinoma** account for over 70% of the data. While the overall accuracy is high (96%), it is biased toward these majority classes. The model struggles significantly with "tail" classes like *Atopic Dermatitis* and *Vascular Lesions* where data is scarce.

14. Future Scope

1. **Data Augmentation for Minority Classes:** Implement synthetic data generation (e.g., GANs) or aggressive augmentation specifically for Vascular Lesions and Dermatitis to improve their detection.
2. **Class Weights:** Retrain the model using `class_weights` in the loss function to penalize misclassification of rare diseases more heavily.
3. **Mobile Application:** Port the Flask logic to a React Native or Flutter app for mobile usage.

15. Conclusion

The Multi-Modal Skin Disease Detection System successfully demonstrates that fusing visual and clinical data yields high-performance diagnostics. With a **96% accuracy rate** and perfect detection of critical conditions like **Melanoma**, the tool holds significant promise for assisting medical professionals. However, addressing the data imbalance remains a critical next step to ensure reliability across all 27 skin conditions.